Formative Assessment Report: Algorithm & Data Structure Analysis

Project: 1 Assignment
Name: Aman Abraha Kasa
Course: DSA in C
Github Repository = Project:1 Assignment

Assignment Question	Source Code File	Data Files Used / Created
Question 1 (Quick Sort)	question-1.c	Reads (`students.txt`) Creates (`sorted_students.txt`)
Question 2 (Bus Route Linked List)	question-2.c	(Runs purely in terminal via user input)
Question 3 (68 Component Binary Tree)	question-3.c	(Runs purely in terminal with built-in array)
Question 4 (Student Records BST)	question-4.c	Reads (raw_students.txt)

Question 1 Analysis: Dynamic Sorting Systems

1. Mathematical Derivations of Quick Sort Running Time

Let T(n) define the time complexity of sorting an array of “n” student records. The partition operation visits each element in the subarray exactly once, introducing a linear workload of Θ(n).

Best Case Scenario

This occurs when the pivot selection consistently splits the array into two perfectly balanced subproblems of size n/2at every level of recursion.

Recurrence Relation:

$$T(n) = 2T(n/2) + \Theta(n)$$

Resolution:

According to Case 2 of the Master Theorem where a=2, b=2, and f(n)=Θ(n)

$$\log_b(a)=\log_2(2)=1$$

Since:

$$f(n)=\Theta(n^{\log_b(a)})$$

the recurrence resolves asymptotically to:

$$T(n)\in O(n\log n)$$

Average Case Scenario

This occurs when pivot selections produce reasonably balanced partitions across recursive levels for an arbitrary unsorted dataset.

Recurrence Relation:

$$T(n) = T(n/10) + T(9n/10) + \Theta(n)$$

Resolution:

Although the partition boundaries may occasionally become asymmetric, the recursive depth remains logarithmic overall. Summing the work performed across all recursion levels gives:

$$T(n)\in O(n\log n)$$

Worst Case Scenario

This occurs when the selected pivot is consistently the minimum or maximum element of the remaining subarray. If the input records are already sorted in ascending or descending order, each partition removes only one element per recursive step.

Recurrence Relation:

$T(n) = T(n-1) + \Theta(n)$

Resolution:

Expanding the recurrence produces the arithmetic series:

$$T(n)= \Sigma_{i=1}^{n} \Theta(i)$$

which simplifies to:

$$T(n)= [n(n+1)]/2$$

Therefore:

$$T(n) \in O(n2)$$

---

2. Critical Evaluation: Quick Sort vs. Insertion Sort

Quick Sort is significantly more appropriate than Insertion Sort for academic analytics management systems due to the following engineering principles:

Dataset Scalability

Insertion Sort operates with quadratic time complexity $O(n^2)$. For a real-world student dataset containing n=100,000 records, Insertion Sort may require approximately:

$$10^{\wedge 10}$$

operations, leading to substantial processing delays.

Quick Sort performs substantially better with average complexity:

$$O(nlogn)$$

requiring approximately:

$$1.7 \times 10^{\wedge 6}$$

operations for the same dataset size, making it highly scalable for large institutional systems.

File-Based Input Realities

Flat-file systems stream records sequentially into memory arrays. Insertion Sort continuously shifts elements during insertion operations, producing large numbers of memory writes and movement operations.

Quick Sort instead processes records in-place using boundary indices and partition-based swapping, significantly reducing insertion overhead and improving efficiency during file-based dataset processing.

Performance Characteristics and Cache Locality

Quick Sort's partitioning strategy accesses memory sequentially through adjacent array indices. This behavior aligns efficiently with modern CPU cache prefetching mechanisms and improves L1/L2 cache utilization.

Although Insertion Sort performs efficiently on very small datasets, repeated element shifting on large arrays increases memory writes and may reduce cache efficiency during large-scale execution.

Consequently, Quick Sort provides substantially better runtime performance, scalability, and memory behavior for modern academic analytics systems handling large student datasets.

Question 2 Analysis: Doubly Linked List Traversal

1. Tail Optimization Analysis

Time Complexity

O(1) (Constant Time)

2. Theoretical Justification

In a standard linked list architecture, appending a new node normally requires a linear traversal from the head pointer to locate the final node. This operation introduces linear time complexity:

O(n)

because the algorithm may need to visit every node in the structure before insertion can occur.

Our navigation system implements an important optimization by maintaining a dedicated pointer called `tail`. The `tail` pointer always stores the exact memory address of the final node in the doubly linked list.

As a result, inserting a new bus stop no longer requires traversal through the list structure. Instead, the program directly accesses the terminal node and performs only constant-time pointer updates regardless of whether the route contains 5 stops or 50,000 stops.

The insertion process is performed using the following operations:

(*tail)->next = newNode;

newNode->prev = *tail;

****tail = newNode;***

These assignments execute in constant time because the program already possesses direct access to the last node in memory.

Therefore, appending a new node in the optimized doubly linked list implementation operates with overall time complexity:

$$O(1)$$

This optimization significantly improves scalability and runtime performance for large transportation route datasets and real-time navigation systems.

Question 3 Analysis: Complete Hierarchical Tree Balancing

1. Structural Insertion Metrics

Time Complexity

$$O(n) \text{ (Linear Time)}$$

---

2. Theoretical Justification

To prevent tree skewing a condition where a binary tree gradually degenerates into a sequential linked-list structure, our implementation maintains a balanced complete binary tree using a level-order insertion strategy.

When a new node is inserted, the algorithm performs a Breadth-First Search (BFS) traversal with the assistance of a queue data structure. The traversal systematically scans nodes from top to bottom and from left to right in order to locate the first available child position.

The algorithm checks whether:

- the left child pointer is empty, or
- the right child pointer is empty.

If both child positions are occupied, the current node's children are inserted into the queue for continued traversal.

This process guarantees that every tree level remains as complete as possible before expanding deeper levels, thereby preserving near-balanced tree characteristics and preventing excessive height growth.

In the worst-case scenario, the insertion operation may require traversing through nearly all existing nodes before locating an available insertion position. For a tree containing nnn nodes, the algorithm may therefore visit up to nnn elements.

Consequently, the insertion operation exhibits worst-case time complexity:

$$O(n)$$

Although the insertion cost is linear, maintaining a complete hierarchical structure significantly improves overall tree balance, traversal efficiency, and search performance compared to skewed binary tree architectures.

Question 4 Analysis: Indexing via Binary Search Trees

1. Search Operation Performance Profiles

The time complexity of searching for records inside a Binary Search Tree (BST) is directly determined by the structural height "h" of the tree.

Operational Profile	**Asymptotic Time Complexity**	**Structural Context**
Best Case	O(1)	The targeted student record matches the root node immediately on the first comparison.
Average Case	O(logn)	Student records are reasonably balanced throughout the tree. The tree height grows

		logarithmically ($h \approx \log_2 n$),reducing the search space by half during each traversal step.
Worst Case	O(n)	The input data is inserted in sorted alphabetical order, causing the BST to degenerate into a sequential linked-list structure. The search operation must then traverse every node linearly(forcing full scan).

2. Conceptual Understanding

Binary Search Tree vs. General Binary Tree

A General Binary Tree imposes no ordering constraints on node placement. It only defines that each parent node may contain at most two child nodes.

A Binary Search Tree (BST), however, enforces a strict structural ordering property:

- all keys in the left subtree must be smaller than the parent node key,
- and all keys in the right subtree must be greater than the parent node key.

This ordering rule enables efficient search operations by systematically eliminating half of the remaining search space during each comparison.

Performance Analysis: Linear File Scan vs. BST Layout

➢ Linear File Search

In a flat-file architecture, locating a student record requires sequentially scanning the file line-by-line from the beginning.

For a dataset containing nnn student records, the average lookup operation performs approximately:

$$n/2$$

comparisons before finding the desired record.

Therefore, linear file searching operates with time complexity:

$$O(n)$$

This becomes increasingly inefficient as dataset sizes grow.

➢ BST Search Optimization

Organizing parsed student records into an in-memory Binary Search Tree enables logarithmic search behavior.

At every node traversal:

- if the search key is smaller than the current node value, traversal continues into the left subtree,
- otherwise traversal proceeds into the right subtree.

This decision process eliminates approximately half of the remaining search space during each step.

Under balanced structural conditions, the search complexity becomes:

$$O(\log n)$$

which provides substantially faster lookup performance than sequential file scanning.

3. Engineering Justification

For a growing university-scale analytics system, a Binary Search Tree provides a significantly more scalable architecture than repeated linear file scanning.

As student enrollment expands into tens or hundreds of thousands of records, sequential disk-based searches introduce major performance bottlenecks due to repetitive file I/O operations.

By parsing the dataset into an in-memory BST structure during application startup, the system performs dramatically faster searches throughout runtime execution.

For example, a balanced BST containing:

$$65536$$

records has approximate search depth:

$$\log_2(65536)=16$$

meaning a lookup operation may require at most only 16 node comparisons under balanced conditions.

This optimization reduces lookup operations from hundreds of thousands of sequential comparisons to only a small number of structural node traversals, delivering highly scalable and efficient institutional data management performance.